## Layer 1: Configuration & Secrets

**.env**

**Holds Actual Secret Credentials**
- File is gitigroned, never committed
- Stores Clio OAuth app credentials (Client ID, Client Secret)
- Sets the redirect URI pointing to Azure Web App
- Sets the Clio API base URL and auth base URL
- file is loaded automatically by config.py when any Python script starts .env.example

**config.py**

**The Single Source of Truth for all Settings**
- Every other Python file imports from here
- It reads from the .env file or from environment variables set by the PowerShell scripts
- If the client ID/secret are missing, it prints an error and stops immediately rather than failing cryptically later

**clio_tokens.json**

**Auto-generated token storage (gitignored)**
- Created by the OAuth flow after you authorize
- Contains access_token, refresh_token, expires_at, created_at
- Read and updated automatically by ClioClient -- you never edit this manually

## Layer 2: Authenication

**config.py**

**Flask web app for the one-time OAuth authorization**

**This is a small web server with 3 routes:**

| Route | What it does |
|---|---|
| GET/ | Landing page with a link to /login |
| GET/login | Builds the Clio authorization URL and redirects you to Clio's login page |
| GET/oauth/callback | Clio redirects back here with an auth code. This route exchanges that code for an access token + refresh token, and saves them to clio_tokens.json |

**Authenication Workflow**

## API Client

**clio_client.py**

**Core Engine that Talks to Clio's API**

**"class ClioClient" - Resuable Client for Clio Manage v4 API, this declared class (i.e., ClioClient) handles all complexity and has four major responsibilities**

1. **Token Management (lines 66 - 171):** before every single API, it checks if the Access Token is expired
   a. If the Access Token is expired it automatically sends the refresh tocken to Clio's "/oauth/token" endpoint and retrieves back a new one and saves it to clie_tokens.json and continues on behind the scences.
   b. if it gets a 401 Bad Request mid-request, which is an edge case scenario where the token has expired between the check and the call, it retries with a fresh token
2. **HTTP Methods (lines 231-259):** Provides get(), post(), patch(), delete() -- the four operations needed/used for Clio's REST API
   a. Each one goes through _request() which handles the token check, rate limiting, and error handling
3. **Rate Limit Handling "*inside _request()*":** If Clio returns a 429 (too many requests), it reads the Retry-After header, waits that many seconds, and retries -- up to 3 attempts
4. **Pagination (lines 263-298):** The get_all() method follows Clio's meta.paging.next URLs to fetch every page of results automatically. Instead of getting 10 or 200 results, it streams through all of them.

## Operations

**operations.py**

**Pre-built Functions for Specific Clio Tasks**
**Each function Takes a ClioClient and Does One Job:**

| Section | Function | Explaination |
|---|---|---|
| READ | list_matters( ), list_contacts( ), list_custom_fields( ), list_document_templates( ), get_matter( ), get_all_matters( ) | Pull data from Clio with sensible default fields |
| UPDATE | update_matter( ), update_custom_field_value( ) | Change a single record |
| BULK | bulk_update_custom_field_from_csv( ), bulk_update_matters_from_csv( ) | Read a CSV file and apply updates to many records at once |
| CREATE | create_contact( ) | Create new resources |
| EXPORT | export_to_json( ), export_to_csv( ) | Save API results to files in data_outputs/ |

**Updating Custom Field -** update_custom_field_value( )

**Clio has Two Different IDs for Custom Fields:**

- **field_def_id** (e.g. 21836420)**:** the field definition, shared across matters
- **value_id** (e.g. "numeric-182750525")**:** the specific value instance on a matter

**Bulk Update of Custom Fields -** bulk_update_custom_field_from_csv()

**Entry Point** (run.py - lines 196 - 208):
**elif cmd in {"bulk-update-cf", "7"} -** prompts for two things:
**1. CSV file path** - the file with your update data
**2. Custom Field ID** - optionally, a single field_def_id to apply to every row (or blank if each row specifies its own)
- This then calls our main function for the Bulk Update Operation

**Operation:** operations.py (lines 284-352):
**Phase 1 -** Load the CSV (lines 298-319)
- Reads every row from the CSV into a list of dicts. Currently expects these columns:
  - matter_id, custom_field_id, value
-

**Matter Date Field Designation:**

**Open Date** - Date the case is opened
**Pending Date** - Date that usually signifies case has been settled and are waiting on the processing of fee's, Date is for the Day that "Pending Date" is entered
**Closed Date** - Date that all items of case has been resolved and closed out